LEZIONE 8
Test Automation e Continuos Testing

Ingegneria del Software 2 – Software Testing

Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

software quality assurance

"… is the systematic activities providing evidence of the fitness for use of the total software product."

- systematic activities → processo
- providing evidence → punti di controllo
- fitness → obiettivi (misurabili) da raggiungere

software quality assurance

"… is the systematic activities providing evidence of the fitness for use of the total software product."

- systematic activities → processo
- providing evidence → punti di controllo
- fitness → obiettivi (misurabili) da raggiungere

software quality assurance

"… is the systematic activities providing evidence of the fitness for use of the total software product."

insieme di piani ed attività per il monitoraggio ed il controllo del processo di sviluppo software

l'obiettivo è assicurare sufficiente confidenza che un prodotto software

- soddisfi le sue specifiche
- sia utilizzabile nel contesto considerato
- venga sviluppato secondo le modalità stabilite

componenti per software quality assurance



componenti per software quality assurance



è solo uno degli aspetti di SQA …

riconsideriamo il precedente semplice esempio

```
int myBuggyFact(int p){




}
```

2^{32} possibili test per cercare
di capire se la procedura
funziona correttamente

riconsideriamo il precedente esempio

```
int myBuggyFact(int p){
```



```
}
```

2^{32} possibili test per cercare di capire se la procedura funziona correttamente



riconsideriamo il precedente esempio

```
int myBuggyFact(int p){
 int result = abs(p);
 if (result > 1){
   int next = result-1;
   result = result*myBuggyFact(next);
 }
 return result;
}
```



2^{32} possibili test per cercare di capire se la procedura funziona correttamente



automated testing – 1

- le attività di test sono svolte principalmente per mezzo di infrastrutture o framework di supporto al software testing
  - automazione nell'esecuzione e nel report di attività di test
  - automazione nella selezione e prioritizzazione dei test da eseguire
  - automazione per valutare la bontà/validità di un dato insieme di casi di test
- si contrappone alle attività svolte da tester umani
  - personale del team di QA, utente finale, etc.
  - questi contesti sono principalmente riferiti come: *manual testing*

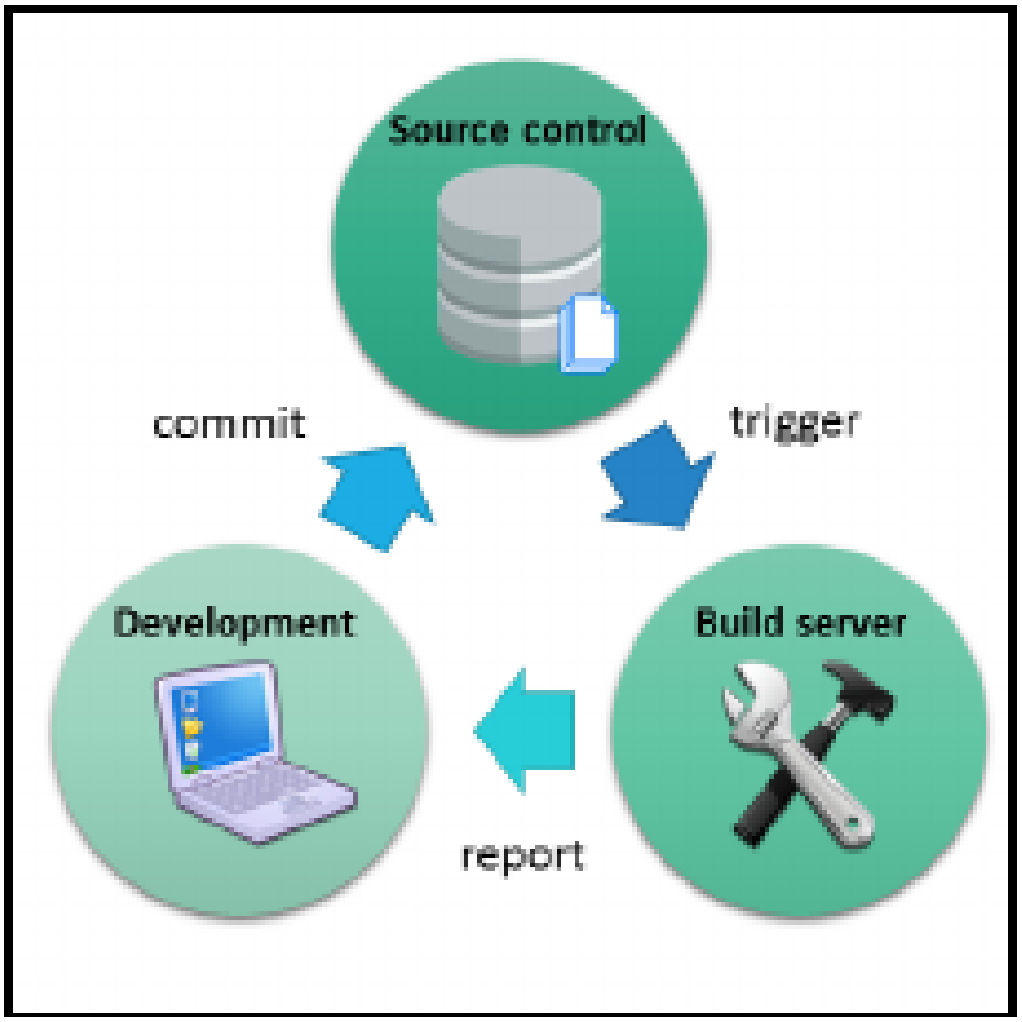
automated testing – 2

- principali obiettivi in AT:
  - migliorare l’efficienza dei processi di software testing
  - migliorare l’efficacia dei processi di software testing
- principali benefici di AT:
  - ridurre i costi
  - ridurre la durata delle fasi di test
  - aumentare la ripetibilità degli esperimenti considerati
  - aumentare l’attendibilità statistica dei processi di software testing
  - migliorare la documentazione delle failures

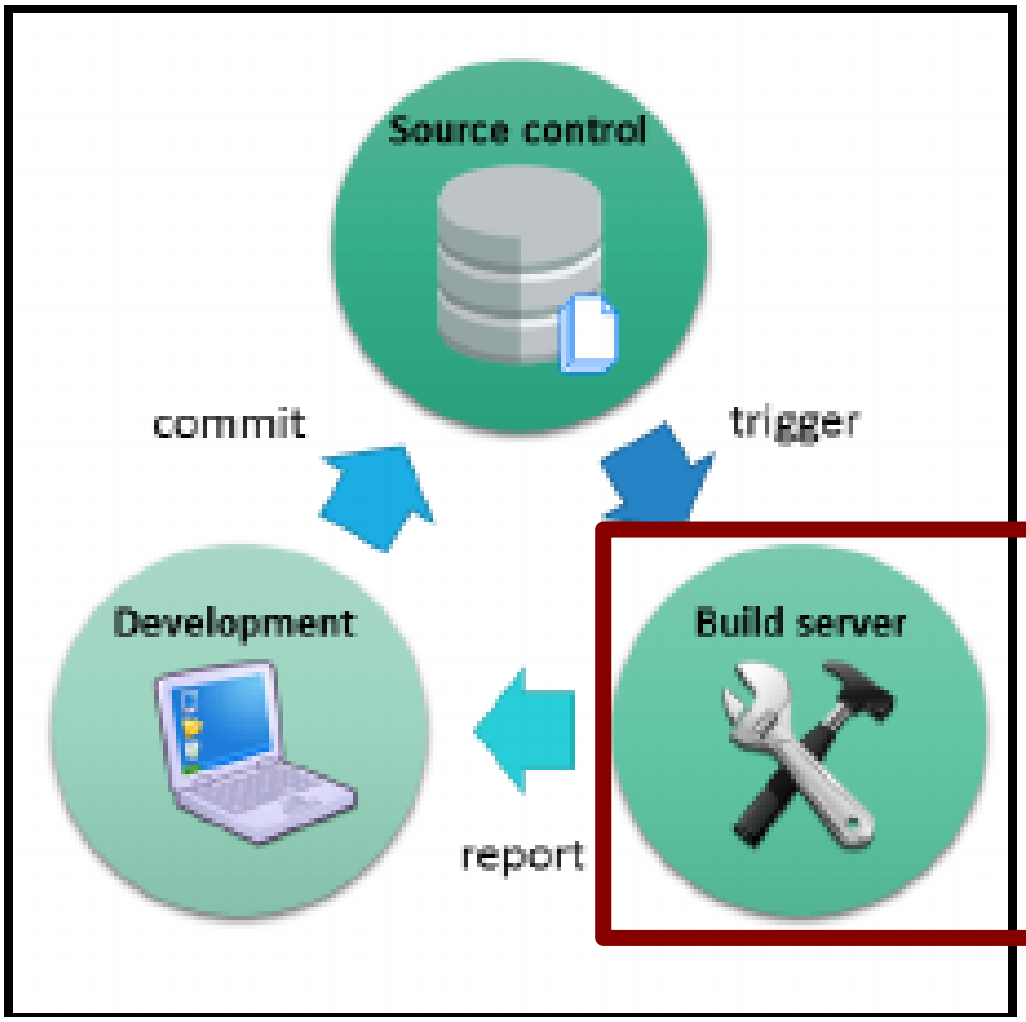
continuous testing

- integrazione dei processi di software testing con
  - ambienti di sviluppo
  - strumenti di build
  - sistemi di versioning
- i test vengono automaticamente eseguiti sulla versione attuale del sistema
  - test eseguiti su configurazioni locali allo sviluppo
  - test eseguiti su ambienti similari agli ambienti di produzione finali
- gli sviluppatori non devono preoccuparsi di avviare esplicitamente fasi di test
  - test eseguiti a seguito di ogni (comprovata) evoluzione della code-base
  - notifiche asincrone rispetto ai risultati dell'esecuzione dei test
    - lo sviluppo non si interrompe!
  - prevedere policy che regolano quali test effettivamente eseguire
    - prioritizzazione, selezione, orchestrazione di casi di test

CI + CT


Source control
commit
trigger
Development
Build server
report

CI + CT

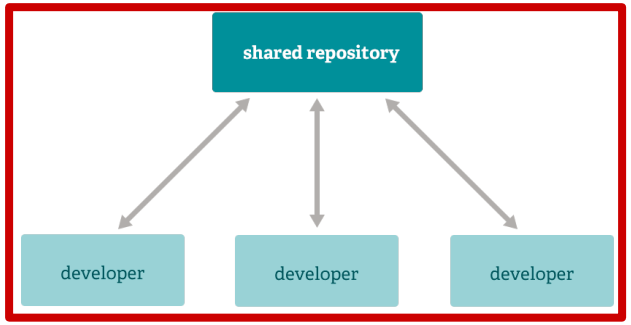


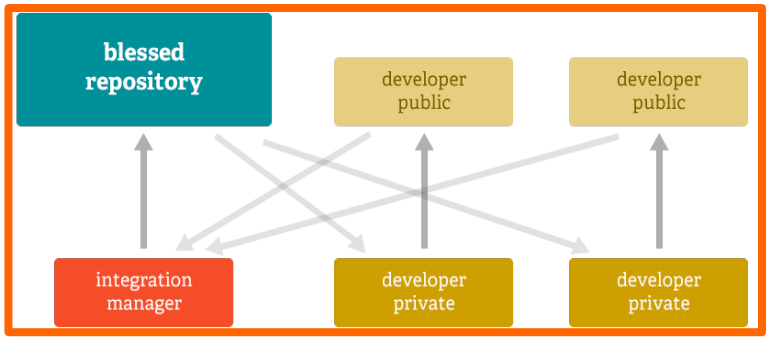
include:
- **configurazione di un ambiente tipo**
- **build del sistema**
- **esecuzione dei casi di test**

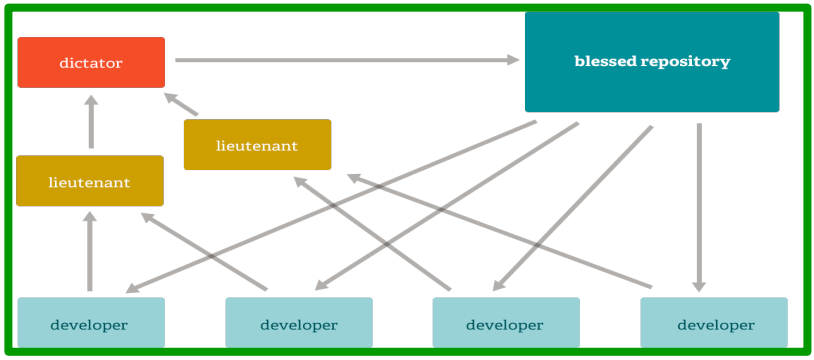
source control – framework

GIT

- VCS open-source e distribuito
  - https://git-scm.com/
  - quanti di voi non hanno avuto esperienza?











automate the build – framework

- MAVEN
  - sistema di build sviluppato dentro Apache SF
    - https://maven.apache.org/
    - concettualmente non troppo diverso da ANT
  - principalmente rivolto ad ambienti Java … ma non solo
  - gestione delle dipendenze
    - esplicita attraverso dichiarazioni di librerie (con versioni) da considerare
    - transitiva: si dichiara solo quello che si usa esplicitamente al resto ci pensa il sistema di build



automate the build – framework

- MAVEN elementi essenziali
  - un progetto MAVEN è strutturato in directory
    - ogni progetto può far riferimento ad una particolare struttura di directory (architype)
    - un progetto può essere diviso in moduli
      - dichiarati internamente o esternamente alla directory del progetto principale
  - un progetto/modulo MAVEN è specificato mediante un descrittore:
    - **`pom.xml`**
      - concettualmente simile a build.xml di ANT
    - tipicamente include: artifactId, groupId, nome, versione, sviluppatori, dipendenze, direttive di build
    - spesso include: direttive di configurazione, direttive di test, direttive di deploy
  - directory di out
    - **`target`**
    - tipicamente i risultati di un build MAVEN vengono strutturati in questa directory
  - MAVEN Central
    - istanze di service registry dove l'istanza locale di MAVEN si collega per acquisire le dipendenze (non disponibili localmente) di un progetto/modulo



automate the build – framework

MAVEN fasi:

- il processo di build è strutturato in fasi
  - le fasi sono concettualmente organizzate in sequenza
  - tipicamente errori nell'esecuzione di una fase comportano il fallimento dell'intero processo di build
- ogni step del processo di build esplicitamente indica la fase di appartenenza
- fasi di default
  - **validate**: validate the project is correct and all necessary information is available
  - **compile**: compile the source code of the project
  - **test**: test the compiled source code using a suitable unit testing framework.
  - **package**: take the compiled code and package it in its distributable format, such as a JAR.
  - **integration-test**: process and deploy the package if necessary into an environment where integration tests can be run
  - **verify**: run any checks to verify the package is valid and meets quality criteria
  - **install**: install the package into the local repository, for use as a dependency in other projects locally
  - **deploy**: done in an integration or release environment, copies the final package to the remote repository
  - **clean**: cleans up artifacts created by prior build
  - **site**: generates site documentation for this project
- è possibile dichiarare ulteriori fasi
  - realisticamente nel nostro corso sarà sufficiente considerarne solo alcune
- ulteriori dettagli su:
  - https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html
  - https://maven.apache.org/guides/getting-started/index.html

